

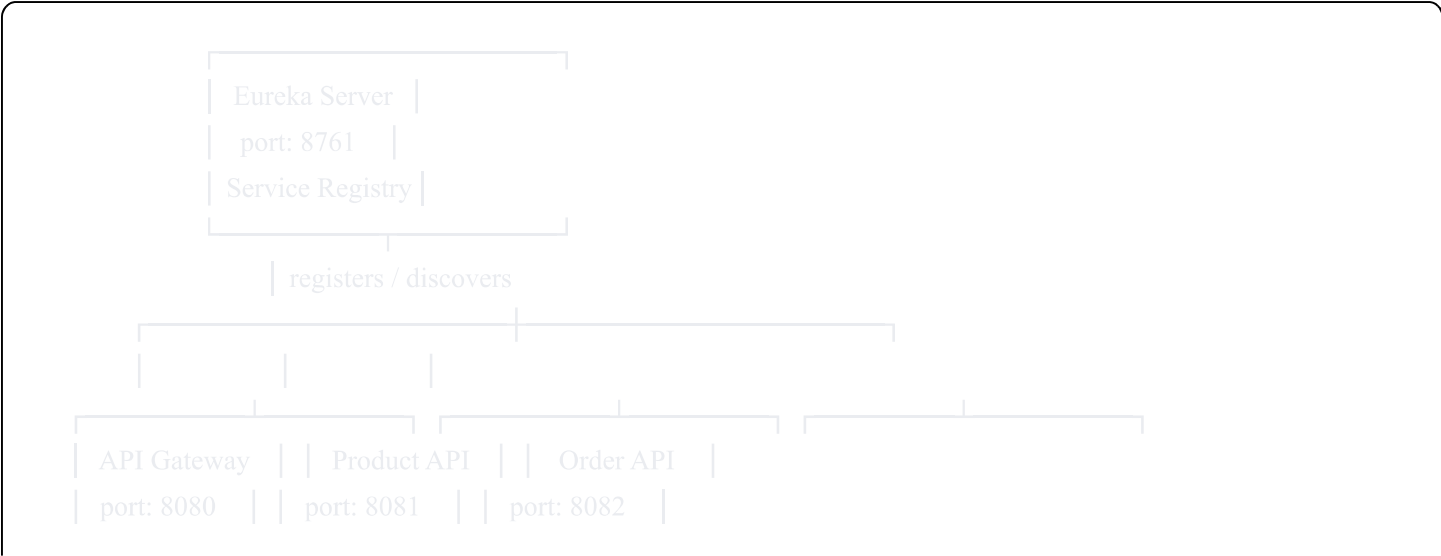
Spring Boot Microservices Project

A complete **e-commerce backend system** built with Spring Boot and Spring Cloud microservices architecture. The system is split into four independent services that communicate with each other over HTTP using service discovery.

Table of Contents

- [Architecture Overview](#)
 - [Services](#)
 - [Technology Stack](#)
 - [Project Structure](#)
 - [Prerequisites](#)
 - [Getting Started](#)
 - [Configuration](#)
 - [API Endpoints](#)
 - [Request & Response Examples](#)
 - [Inter-Service Communication](#)
 - [Design Principles](#)
 - [Error Handling](#)
 - [What's Next](#)
-

Architecture Overview





All client requests go through the **API Gateway** on port `8080`. The gateway uses **Eureka** to discover where each service is running and routes requests accordingly. Each service has its own dedicated database — no sharing.

Services

1. Eureka Server — `port 8761`

The **service registry**. Every service registers itself here on startup and looks up other services here when it needs to communicate. Think of it as the phone book of the system.

- Built with: `spring-cloud-starter-netflix-eureka-server`
- Dashboard: `http://localhost:8761`

2. API Gateway — `port 8080`

The **single entry point** for all client requests. No client ever calls `product-api` or `order-api` directly — everything goes through the gateway. It examines the request path and routes it to the correct downstream service using Eureka for service discovery.

- Built with: `spring-cloud-starter-gateway` (WebFlux / reactive)
- Routes `/api/products/**` → `product-api`
- Routes `/api/orders/**` → `order-api`

3. Product API — `port 8081`

Manages everything related to **products**. Has its own dedicated `product_db` that no other service can access directly. Exposes full CRUD REST endpoints.

- Built with: Spring Web, Spring Data JPA, MySQL, Lombok, Eureka Client
- Database: `product_db`
- Manages: name, description, price, quantity

4. Order API — port 8082

Manages everything related to **orders**. When a client places an order, this service calls `product-api` via **Feign Client** to validate the product and check stock availability, then saves the order to its own `order_db`.

- Built with: Spring Web, Spring Data JPA, MySQL, Lombok, OpenFeign, Eureka Client
- Database: `order_db`
- Manages: productId, productName, quantity, unitPrice, totalPrice, status
- Communicates with: `product-api` via Feign

Technology Stack

Technology	Version	Purpose
Java	21	Programming language
Spring Boot	3.5.11	Application framework
Spring Cloud	2025.0.1	Microservices toolkit
Spring Cloud Gateway	4.x	API Gateway (reactive)
Netflix Eureka	-	Service discovery & registration
Spring Data JPA	-	Database access layer
OpenFeign	-	Declarative HTTP client for inter-service calls
MySQL	8.x	Relational database
Lombok	-	Reduces boilerplate code
Maven	3.9.x	Build and dependency management

Project Structure

Each service follows the same layered package structure:

```
service-name/  
└── src/main/java/com/pathum/servicename/  
    ├── ServiceApplication.java    ← entry point  
    │  
    ├── controller/              ← HTTP in/out only, no business logic  
    │   └── EntityController.java  
    │  
    ├── service/                 ← interface defines the contract  
    │   ├── EntityService.java  
    │   └── impl/  
    │       └── EntityServiceImpl.java ← business logic lives here  
    │  
    ├── repository/              ← talks to database only  
    │   └── EntityRepository.java  
    │  
    ├── entity/                  ← maps to database table  
    │   └── Entity.java  
    │  
    ├── dto/  
    │   ├── request/             ← what client sends in (validated)  
    │   │   └── EntityRequestDto.java  
    │   ├── response/            ← what you send back (controlled)  
    │   │   └── EntityResponseDto.java  
    │  
    ├── mapper/                  ← converts entity ↔ dto  
    │   └── EntityMapper.java  
    │  
    ├── exception/               ← centralised error handling  
    │   ├── EntityNotFoundException.java  
    │   └── GlobalExceptionHandler.java  
    │  
    └── client/                  ← Feign clients (order-api only)  
        └── ProductClient.java
```

Prerequisites

Make sure you have the following installed before running the project:

- Java 21
- Maven 3.9+
- MySQL 8.x or MariaDB

- Git
-

Getting Started

Step 1 — Create Databases

Open your MySQL client and run:

```
sql

CREATE DATABASE product_db;
CREATE DATABASE order_db;
```

Step 2 — Configure Database Credentials

Update the `application.properties` in both `product-api` and `order-api` with your MySQL credentials:

```
properties

spring.datasource.username=root
spring.datasource.password=your_password
```

Step 3 — Start Services in Order

Always start services in this exact order:

```
bash

# 1. Start Eureka Server first — everything depends on it
cd eureka-server
./mvnw spring-boot:run

# 2. Start Product API
cd product-api
./mvnw spring-boot:run

# 3. Start Order API
cd order-api
./mvnw spring-boot:run

# 4. Start API Gateway last
cd api-gateway
./mvnw spring-boot:run
```

Step 4 — Verify Everything is Running

Open the Eureka dashboard at `http://localhost:8761`. You should see all three services registered:

```
api-gateway    UP
product-api    UP
order-api      UP
```

Configuration

Eureka Server — `application.properties`

```
properties

spring.application.name=eureka-server
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.server.wait-time-in-ms-when-sync-empty=0
```

API Gateway — `application.properties`

```
properties

spring.application.name=api-gateway
server.port=8080

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.cloud.gateway.server.webflux.routes[0].id=product-api
spring.cloud.gateway.server.webflux.routes[0].uri=lb://product-api
spring.cloud.gateway.server.webflux.routes[0].predicates[0]=Path=/api/products/**

spring.cloud.gateway.server.webflux.routes[1].id=order-api
spring.cloud.gateway.server.webflux.routes[1].uri=lb://order-api
spring.cloud.gateway.server.webflux.routes[1].predicates[0]=Path=/api/orders/**
```

`lb://` means load balanced — the gateway asks Eureka for the actual address.

Product API — `application.properties`

```
properties
```

```
spring.application.name=product-api
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/product_db
spring.datasource.username=root
spring.datasource.password=your_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Order API — application.properties

properties

```
spring.application.name=order-api
server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/order_db
spring.datasource.username=root
spring.datasource.password=your_password
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

API Endpoints

All endpoints are accessible via the API Gateway on port **8080**.

Product API

Method	Endpoint	Description
POST	/api/products	Create a new product

Method	Endpoint	Description
GET	/api/products	Get all products
GET	/api/products/{id}	Get product by ID
GET	/api/products/search?name=x	Search products by name
PUT	/api/products/{id}	Update a product
DELETE	/api/products/{id}	Delete a product

Order API

Method	Endpoint	Description
POST	/api/orders	Place a new order
GET	/api/orders	Get all orders
GET	/api/orders/{id}	Get order by ID
GET	/api/orders/status/{status}	Get orders by status
PATCH	/api/orders/{id}/status?status=x	Update order status
DELETE	/api/orders/{id}	Cancel an order

Order Status Values

PENDING → initial status when order is placed

CONFIRMED → order has been confirmed

CANCELLED → order has been cancelled

Request & Response Examples

Create a Product

http

POST http://localhost:8080/api/products

Content-Type: application/json

```
{  
  "name": "Gaming Laptop",  
  "description": "High performance laptop",  
  "price": 999.99,  
  "quantity": 10  
}
```

Response **201 Created**:

json

```
{  
  "id": 1,  
  "name": "Gaming Laptop",  
  "description": "High performance laptop",  
  "price": 999.99,  
  "quantity": 10,  
  "createdAt": "2026-02-27T10:00:00",  
  "updatedAt": "2026-02-27T10:00:00"  
}
```

Place an Order

http

POST http://localhost:8080/api/orders

Content-Type: application/json

```
{  
  "productId": 1,  
  "quantity": 2  
}
```

Response **201 Created**:

json

```
{
  "id": 1,
  "productId": 1,
  "productName": "Gaming Laptop",
  "quantity": 2,
  "unitPrice": 999.99,
  "totalPrice": 1999.98,
  "status": "PENDING",
  "createdAt": "2026-02-27T10:01:00",
  "updatedAt": "2026-02-27T10:01:00"
}
```

Update Order Status

http

PATCH http://localhost:8080/api/orders/1/status?status=CONFIRMED

Get Orders by Status

http

GET http://localhost:8080/api/orders/status/PENDING

GET http://localhost:8080/api/orders/status/CONFIRMED

GET http://localhost:8080/api/orders/status/CANCELLED

Search Products by Name

http

GET http://localhost:8080/api/products/search?name=laptop

Inter-Service Communication

When a client places an order, the following flow happens behind the scenes:

1. Client sends POST /api/orders to API Gateway (port 8080)
2. API Gateway receives request
 - asks Eureka "where is order-api?"
 - Eureka responds "port 8082"
 - Gateway forwards request to order-api
3. OrderController receives and validates request
 - calls OrderService
4. OrderServiceImpl runs business logic
 - calls ProductClient.getProductById(productId) via Feign
5. Feign Client asks Eureka "where is product-api?"
 - Eureka responds "port 8081"
 - Feign makes GET /api/products/{id} to product-api
6. ProductController in product-api handles the call
 - fetches product from product_db
 - returns ProductResponseDto
7. Back in OrderServiceImpl
 - validates stock availability
 - calculates totalPrice = unitPrice × quantity
 - saves order to order_db with status PENDING
8. OrderResponseDto returned all the way back to client

Feign Client (order-api)

```
java

@FeignClient(name = "product-api")
public interface ProductClient {

    @GetMapping("/api/products/{id}")
    ProductResponseDto getProductById(@PathVariable Long id);
}
```

The `name = "product-api"` must exactly match `spring.application.name` in product-api. Eureka uses this name to locate the service.

Design Principles

Single Responsibility

Each service does exactly one thing. Product service manages products, order service manages orders. They do not overlap.

Database Per Service

```
product-api → product_db (private, no other service touches it)
order-api   → order_db   (private, no other service touches it)
```

Data is shared only through API calls, never through direct database access.

Loose Coupling

Services never hardcode each other's URLs. They use Eureka for dynamic discovery which means services can be moved, scaled, or replaced without changing other services.

DTO Pattern

```
Entity      → your database structure (never leaves the service layer)
RequestDto  → what the client sends in (with validation)
ResponseDto → what you send back (you control what's exposed)
Mapper      → converts between them (single responsibility)
```

Centralised Entry Point

All traffic goes through the API Gateway. This gives you one place to later add cross-cutting concerns like authentication, rate limiting, and logging without touching individual services.

Error Handling

All services return consistent error responses via `GlobalExceptionHandler`.

Not Found — `404`

```
json
```

```
{
  "timestamp": "2026-02-27T10:00:00",
  "status": 404,
  "message": "Product not found with id: 99"
}
```

Validation Failed — 400

json

```
{
  "timestamp": "2026-02-27T10:00:00",
  "status": 400,
  "message": "Validation failed",
  "errors": {
    "name": "Product name is required",
    "price": "Price must be greater than 0"
  }
}
```

Conflict — 409

json

```
{
  "timestamp": "2026-02-27T10:00:00",
  "status": 409,
  "message": "Product with name 'Gaming Laptop' already exists"
}
```

Insufficient Stock — 400

json

```
{
  "timestamp": "2026-02-27T10:00:00",
  "status": 400,
  "message": "Insufficient stock. Available: 10"
}
```

What's Next

Features that can be added to extend this project:

Short Term

- Stock deduction in product-api when an order is placed
- Spring Security with JWT authentication on the API Gateway
- Circuit Breaker with Resilience4J to handle service failures gracefully

Medium Term

- Docker and Docker Compose to containerise all services
- Spring Cloud Config Server to centralise all configuration files
- Distributed tracing with Zipkin to trace requests across services

Long Term

- Apache Kafka for async event-driven communication between services
- Payment Service for processing payments
- Notification Service for sending order confirmation emails
- Kubernetes deployment for production orchestration

Author

Built by **Pathum** as a learning project for Spring Boot Microservices architecture.